Collections, Generics, Wrapper Classes and Multithreading in Java

Object-Oriented Programming in Java — Week 8

This chapter answers two practical questions. First, how do we store and manage a *group* of objects when we cannot know in advance how many there will be? Second, how do we let a program do several things at once without the threads corrupting each other's data? The first question leads to the **Collections Framework**, **generics** and **wrapper classes**. The second leads to **multithreading**, **synchronization** and **inter-thread communication**.

> **How to read the examples:** every program is a separate, self-contained file. Output shown is what the code prints on a standard JVM. Where the language does not guarantee an order (hash-based collections, thread interleaving), this is stated explicitly.

Index

- 1. Learning Objectives — what you should be able to do before the tea gets cold
- 2. Why Collections Exist — arrays, and why 100 slots is never the right number
- 3. The Java Collections Framework — one family tree to rule all containers
- 4. The Four Main Interfaces — list, set, queue, map: four personalities
- 5. The List Family — ordered, duplicate-friendly, index-loving
- 6. The Set Family — the exclusive guest list
- 7. The Queue Interface — first come, first served, no cutting
- 8. The Map Interface — look things up by key, not by position
- 9. Iterators — a bookmark that walks through any collection
- 10. Choosing the Right Collection — picking the right container for the job
- 11. Generics — labelling the box before you fill it
- 12. Wrapper Classes — gift-wrapping primitives so collections accept them
- 13. Multithreading and the Thread Life Cycle — many cooks, one kitchen
- 14. Creating and Controlling Threads — hiring workers and giving them job cards
- 15. Thread Synchronization — one person at the counter at a time
- 16. Inter-thread Communication — wait, notify, and the producer-consumer handshake
- 17. Summary and Lecture Exercises — recap and homework
- Practice Questions — five problems that bite back

---

1. Learning Objectives

Collections and generics

- Understand the Java Collections Framework (JCF) and its architecture.
- Select a suitable collection class according to application requirements (array-like access, uniqueness, ordering, FIFO processing, key-value lookup).
- Implement and use `ArrayList`, `LinkedList`, `HashSet`, `TreeSet` and `HashMap`.
- Traverse collections using an `Iterator` (and `ListIterator`).
- Understand why generics matter for type safety and code reuse.
- Use wrapper classes for primitive data when working with collections and generics.

Multithreading

- Understand threads and concurrent programming.
- Explain the thread life cycle and the thread states.
- Create and manage threads in Java.
- Use synchronization to avoid data inconsistency when threads access the same variable.
- Understand the inter-thread communication mechanism.

2. Why Collections Exist

The problem. A college office wants to store the roll numbers of students who register for an elective. Nobody knows in advance how many will register. Before collections, the standard tool was an array.

An array is declared with a fixed size. In the statement on the right, `int arr[]` declares a reference variable that can refer to an integer array, and `new int[100]` creates the array object with exactly 100 integer slots. The size is decided at creation and can never change afterwards.

```
int arr[] = new int[100];
// Same meaning, preferred style:
// int[] arr = new int[100];
```

The analogy. An array is a printed attendance register with exactly 100 lines. If 101 students arrive you must buy a new register and copy everything across. A collection is a register that adds pages by itself.

Problems with the traditional array	What collections provide
Fixed size	Dynamic size: grows and shrinks as the application needs
Cannot store different object types (an `int[]` holds only `int` values)	Ready-made data structures (list, set, queue, map)
No built-in methods for sorting and searching	Efficient built-in operations

> **Note for Review:** An array object itself has no sort or search methods, only a `length` field. The helper class `java.util.Arrays` does offer `sort` and `binarySearch`, but even that cannot make an array grow. The fixed size plus the lack of ready-made structures is what collections solve.

3. The Java Collections Framework

The **Java Collections Framework (JCF)** is a unified architecture for storing and manipulating groups of objects. It provides three things:

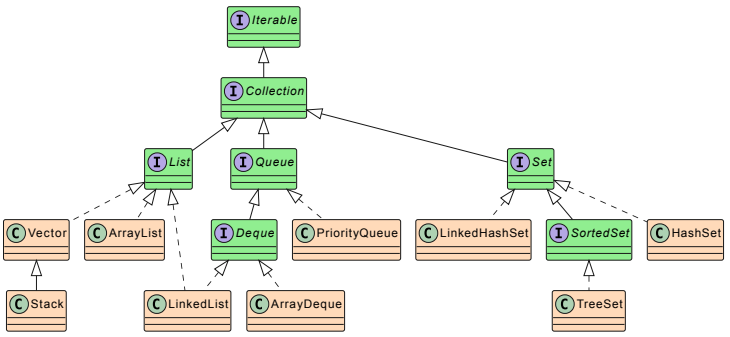
- **Interfaces:** the contracts describing what a kind of collection can do.
- **Implementation classes:** the actual, ready-to-use data structures.
- **Algorithms:** ready-made operations such as searching and sorting.

The framework lives in the `java.util` package, which is why collection programs begin with an import.

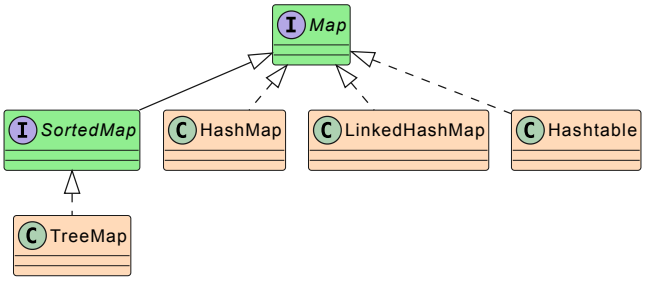
Import statement	Meaning
`import java.util.*;`	Makes every public class of `java.util` available (`*` means "all")
`import java.util.ArrayList;`	Imports only the one class you name

Interface versus class. An *interface* says *what* a collection must be able to do. A *class* says *how* it does it. `List` is like the job description "attendance register"; `ArrayList` and `LinkedList` are two different ways of building one. You cannot create an object of an interface (`new List<>()` is invalid), but you can create objects of its classes and then perform any operation on them: adding, printing, searching and so on.

The diagram shows the `Collection` side of the framework. `Iterable` sits at the top, so every collection can be traversed. `Collection` splits into `List`, `Queue` and `Set`. Green boxes are interfaces and orange boxes are classes. A solid arrow means *extends* (interface to interface or class to class) and a dotted arrow means *implements* (class to interface).



`Map` has its own, separate hierarchy: it is **not** a subtype of `Collection`, because it stores key-value pairs rather than single elements. `HashMap`, `LinkedHashMap` and `Hashtable` are classes implementing `Map`, while `SortedMap` is an interface that extends it and is implemented by `TreeMap`.



> **Note for Review:** In the lecture's architecture figure the classes marked as the core ones are `Iterable`, `Collection`, `List`, `Set`, `ArrayList`, `LinkedList`, `HashSet`, `LinkedHashSet` and `TreeSet`. Supplementary detail: in the actual JDK, `LinkedHashSet` extends `HashSet` and `LinkedHashMap` extends `HashMap`; the diagrams above follow the simplified lecture picture.

4. The Four Main Interfaces

Different textbooks group the framework as *list, set, queue* or as *list, set, map*. The ideas are the same. In practice there are four main interfaces, and the one you pick decides how your data behaves.

Interface	Insertion order	Duplicates	Core idea	Examples
`List`	Maintained	Allowed	Index-based access, like an array	`ArrayList`, `LinkedList`, `Vector`, `Stack`
`Set`	Not maintained (except `LinkedHashSet`)	Not allowed	Useful for unique collections	`HashSet`, `TreeSet`
`Queue`	Processing order (FIFO)	Allowed	Elements waiting to be processed; used in scheduling and resource management	`PriorityQueue`, `LinkedList`
`Map`	Not guaranteed	Keys unique, values may repeat	Stores data as key → value	`HashMap`, `TreeMap`

> **Note for Review: FIFO** means *First In, First Out*: the element that arrived first leaves first. Its opposite, **LIFO** (last in, first out), is how a stack behaves. Supplementary clarification: a `PriorityQueue` does not follow strict arrival order; it serves the element with the highest priority (by default the smallest value) first. Strict FIFO behaviour is what `LinkedList` and `ArrayDeque` give when used as queues.

5. The List Family

5.1 The `List` interface

The problem. A coaching class keeps an attendance register. Order matters (first student, second student...), and two different students may share a name, so duplicates must be allowed. We also want to say "give me the student on line 3".

A `List` therefore:

- maintains **insertion order**,
- allows **duplicate elements**,
- allows **index-based access** (positions start at `0`).

The program creates a list of names and adds "Rahul" twice. Because a list allows duplicates, both copies are stored in the order they were added. Printing the list shows all three elements inside square brackets.

```
import java.util.*;

class ListDemo {
  public static void main(String args[]) {
    List<String> names =
        new ArrayList<>();
    names.add("Rahul");
    names.add("Amit");
    names.add("Rahul");
    System.out.println(names);
  }
}
```

Output:

```
[Rahul, Amit, Rahul]
```

Syntax breakdown of the key line

```
List<String> names = new ArrayList<>();
```

Part	Meaning
`List`	The *interface* type of the reference variable. Through `names` you can use only the `List` operations
`<String>`	Type argument: this list may hold only `String` objects
`names`	Name of the reference variable
`=`	Assigns the reference of the new object to the variable
`new ArrayList<>()`	Creates an `ArrayList` object; `()` calls its constructor
`<>` (diamond)	Tells the compiler to infer the type argument (`String`) from the left side
`;`	Ends the statement

The declaration and the creation are really two ideas. `List<String> names;` only declares a reference variable. `new ArrayList<>()` creates the object, and `=` stores its reference in the variable.

Execution trace

Step	Operation	Result
1	Create an `ArrayList` object, store its reference in `names`	`[]` (empty)
2	`names.add("Rahul")`	`[Rahul]`
3	`names.add("Amit")`	`[Rahul, Amit]`
4	`names.add("Rahul")`	`[Rahul, Amit, Rahul]` (duplicate accepted)
5	`System.out.println(names)`	Prints the list's text form

> **Note for Review:** The variable's type (`List` , known at compile time) and the object's type (`ArrayList` , existing at run time) are different. Writing the interface on the left lets you swap `ArrayList` for `LinkedList` later by changing only the right-hand side. Also, `String args[]` in `main` is legal C-style array syntax; `String[] args` is the more common Java style and means the same.

5.2 `ArrayList`

`ArrayList` is a **resizable array implementation** of the `List` interface.

- **Dynamic size:** it grows as elements are added.
- **Maintains insertion order.**
- **Allows duplicates.**
- **Fast random access:** `get(i)` jumps straight to position `i` .

Declaration pattern

```
ArrayList<Type> list = new ArrayList<>();
```

`Type` must be a class (for numbers, a wrapper such as `Integer` ; see 12. Wrapper Classes).

This program stores three integers, prints the list, then removes the element at index `1` . Indexes start at 0, so index 1 is the *second* position and the value 20 disappears. The elements after the removed one shift one place to the left.

```
import java.util.*;

class ArrayListDemo {
  public static void main(String args[]) {
    ArrayList<Integer> numbers =
        new ArrayList<>();
    numbers.add(10);
    numbers.add(20);
    numbers.add(30);
    System.out.println(numbers);
    numbers.remove(1);
    System.out.println(numbers);
  }
}
```

Output:

```
[10, 20, 30]
[10, 30]
```

Step	Operation	List after the step	Positions
1	`add(10)` , `add(20)` , `add(30)`	`[10, 20, 30]`	10 at 0, 20 at 1, 30 at 2
2	`remove(1)`	`[10, 30]`	30 moves from position 2 to 1

> **Note for Review:** `remove(1)` removes the element **at index 1**, not the value `1` . `ArrayList<Integer>` has two methods, `remove(int index)` and `remove(Object o)` . A plain `int` argument selects the index version. To delete the *value* 1 you must pass an object: `numbers.remove(Integer.valueOf(1))` . Also, Java prints list elements separated by a comma and a space, as in `[10, 20, 30]` .

Common `ArrayList` methods

Method	Description
`add()`	Add element
`remove()`	Remove element
`get()`	Access element
`set()`	Replace element
`size()`	Number of elements
`clear()`	Remove all elements

```
import java.util.*;

class ArrayListMethods {
  public static void main(String args[]) {
    ArrayList<Integer> marks =
        new ArrayList<>();
    marks.add(72);
    marks.add(85);
    marks.add(64);
    System.out.println(marks.get(1));
    marks.set(2, 70);
    System.out.println(marks);
    System.out.println(marks.size());
    marks.remove(Integer.valueOf(85));
    System.out.println(marks);
    marks.clear();
    System.out.println(marks.size());
  }
}
```

This supplementary example exercises every method in the table using student marks. `get(1)` reads, `set(2, 70)` replaces the value at index 2, `size()` counts, `remove(Integer.valueOf(85))` deletes by value, and `clear()` empties the list. Using an index outside the valid range (for example `get(5)` on a three-element list) throws an `IndexOutOfBoundsException` at run time.

Output:

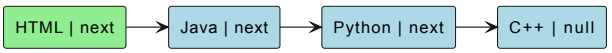
```
85
[72, 85, 70]
3
[72, 70]
0
```

> **Note for Review (supplementary):** Internally an `ArrayList` keeps its elements in an ordinary array. When that array is full, a bigger one is allocated and the elements are copied across. That is why reading by index is fast, while inserting or deleting in the middle is slower (the later elements must shift).

5.3 `LinkedList`

A `LinkedList` stores its elements as **nodes connected through links**. Each node contains **data plus the address of the next node**. It maintains order and allows duplicates. Its selling point is **fast insertion and deletion**: to add at the front you only create a node and re-point a link, with no shifting of other elements.

After `addFirst("HTML")` the list is a chain of four nodes. Each node holds its data and a link to the next node, and the last node's link is `null`. The new green node became the first node without any other node having to move.



> **Note for Review (supplementary):** The lecture's model is a *singly* linked node (data plus next address). Java's actual `LinkedList` is *doubly* linked: each node also stores the previous node's address. The idea is the same.

This program builds a list of three languages, prints it, then uses `addFirst` to insert "HTML" at the very beginning. `LinkedList` offers methods such as `addFirst` that the general `List` interface does not, which is why the variable is declared with the class type here.

```
import java.util.*;

class LinkedListDemo {
  public static void main(String args[]) {
    LinkedList<String> list =
        new LinkedList<>();
    list.add("Java");
    list.add("Python");
    list.add("C++");
    System.out.println(list);
    list.addFirst("HTML");
    System.out.println(list);
  }
}
```

Output:

```
[Java, Python, C++]
[HTML, Java, Python, C++]
```

`ArrayList` versus `LinkedList`

Aspect	`ArrayList`	`LinkedList`
Internal structure	Dynamic (resizable) array	Linked nodes
Searching / reading by index	Faster: $O(1)$ for `get(i)`	Slower: $O(n)$, must walk the links
Insertion / deletion	Slower in the middle (elements shift)	Faster: just re-link nodes
Memory overhead	Less	More (every node stores a link)

> **Note for Review:** The $O(\cdot)$ figures are supplementary. Insertion in a `LinkedList` is only cheap once you are already at the right node; *finding* the middle position still costs $O(n)$. The quick insertion benefit is clearest at the front or end.

> **Note for Review (supplementary):** `Vector` is the older, synchronized relative of `ArrayList`, and `Stack` (last-in, first-out) extends `Vector`. Both belong to the `List` family named in the lecture.

6. The Set Family

The problem. A hostel mess wants a list of the *distinct* students who ate breakfast today. Some students swiped their cards twice. Storing everything in a list would count them twice. We need a container that silently refuses duplicates.

A `Set`:

- **does not allow duplicate elements,**
- **does not maintain insertion order** (the exception is `LinkedHashSet`, which does),
- is useful for **unique collections**.

6.1 `HashSet`

`HashSet` stores unique elements using **hashing**. Its features are: no duplicates, **no guaranteed order**, and it allows **one `null` value**.

The program adds "Java", "Python" and then "Java" again. The second "Java" is ignored, so the set holds only two elements. Note that `<String>` fixes the element type: this set can hold only strings.

```
import java.util.*;

class HashSetDemo {
  public static void main(String args[]) {
    HashSet<String> set =
        new HashSet<>();
    set.add("Java");
    set.add("Python");
    set.add("Java");
    System.out.println(set);
  }
}
```

Output:

```
[Java, Python]
```

> **Note for Review:** The *order* `[Java, Python]` is what this JVM happens to print; a `HashSet` does not promise any order, so never write code that depends on it. Supplementary: to decide whether an element is a duplicate, a `HashSet` uses `hashCode()` and `.equals()`, not `==`, so two different `String` objects with the same text count as duplicates.

The duplicate "Java" is ignored.

Step	Operation	Result
1	`add("Java")`	Added; returns `true`
2	`add("Python")`	Added; returns `true`
3	`add("Java")`	Already present, set unchanged; returns `false`
4	`println(set)`	Prints two elements

6.2 `TreeSet`

`TreeSet` stores elements in **sorted order**. It allows no duplicates, keeps its elements sorted (ascending for numbers) and uses a **Red-Black tree** internally (a self-balancing search tree).

The program inserts 50, 10 and 30, in that order. A `TreeSet` places each new value in its sorted position, so printing shows ascending order regardless of insertion order. This sorted output is exactly what a `HashSet` does not guarantee.

Output:

```
[10, 30, 50]
```

```
import java.util.*;

class TreeSetDemo {
  public static void main(String args[]) {
    TreeSet<Integer> set =
        new TreeSet<>();
    set.add(50);
    set.add(10);
    set.add(30);
    System.out.println(set);
  }
}
```

> **Note for Review (supplementary):** Elements of a `TreeSet` must be comparable with one another (`Integer` and `String` are). A `TreeSet` using natural ordering does not accept `null`.

6.3 `HashSet` versus `TreeSet`

`HashSet`	`TreeSet`
No ordering	Sorted order
Faster operations	Slower than `HashSet`
Uses hashing	Uses a tree structure

Supplementary: for n elements, `add`, `remove` and `contains` take about $O(1)$ time on average in a `HashSet` and $O(\log n)$ in a `TreeSet`. `LinkedHashSet` sits between them: unique elements, but kept in insertion order.

7. The Queue Interface

The problem. A customer-service desk handles complaint tickets. The ticket that arrived first must be handled first. A list lets you remove anything from anywhere, which is not the discipline we want.

The analogy. A queue at a bank counter: people join at the back and are served from the front.

`Queue` follows **FIFO (First In, First Out)** and represents elements waiting for processing. It is used in **scheduling** and **resource management**. `PriorityQueue` and `LinkedList` are examples of queue implementations.

Method	Description
`add()`	Insert element (at the back)
`remove()`	Remove element (from the front)
`peek()`	View the first element without removing it

The program uses a `LinkedList` through a `Queue` reference. The reference type is `Queue`, so only queue operations are visible through `q`, while the object is a `LinkedList`. After three insertions, `peek()` shows the front element and `remove()` removes it.

```
import java.util.*;

class QueueDemo {
  public static void main(String args[]) {
    Queue<String> q =
        new LinkedList<>();
    q.add("A");
    q.add("B");
    q.add("C");
    System.out.println(q);
    System.out.println(q.peek());
    q.remove();
    System.out.println(q);
  }
}
```

Output:

```
[A, B, C]
A
[B, C]
```

Step	Operation	Queue (front → back)	Printed
1	`add("A")`, `add("B")`, `add("C")`	`A, B, C`	
2	`println(q)`	`A, B, C`	`[A, B, C]`
3	`q.peek()`	`A, B, C` (unchanged)	`A`
4	`q.remove()`	`B, C` ("A" was first in, so first out)	
5	`println(q)`	`B, C`	`[B, C]`

> **Note for Review (supplementary):** On an empty queue, `remove()` throws a `NoSuchElementException`, whereas `peek()` and `poll()` return `null`. Because a `LinkedList` can be both a `List` and a `Queue`, the variable's declared type decides which methods you may call.

8. The Map Interface

The problem. A college office needs to find a student's name from the roll number. A list identifies items by *position* (`0, 1, 2...`), but roll numbers such as 101, 102, 103 are not positions. We need a container where *we* choose the label.

The analogy. A phone directory: you look up a name (the **key**) and get a number (the **value**).

A `Map` stores data as **key → value** pairs.

- **Keys must be unique.**
- **Values can be duplicated.**
- Examples: `HashMap`, `TreeMap`.

8.1 `HashMap`

`HashMap` stores data in key-value pairs, for example `101 → Rahul` and `102 → Amit`. Its features: **unique keys**, **duplicate values allowed**, **no ordering**, and **one `null` key** allowed.

The program creates a map from `Integer` keys to `String` values, inserts three entries with `put`, prints the whole map and then retrieves one value with `get`. Two type arguments are needed now: first the key type, then the value type.

Output:

```
{101=Rahul, 102=Amit, 103=Neha}
Rahul
```

```
import java.util.*;

class HashMapDemo {
  public static void main(String args[]) {
    HashMap<Integer, String> map =
        new HashMap<>();
    map.put(101, "Rahul");
    map.put(102, "Amit");
    map.put(103, "Neha");
    System.out.println(map);
    System.out.println(map.get(101));
  }
}
```

Part	Meaning
`HashMap<Integer, String>`	Keys are `Integer`, values are `String` (primitives like `int` cannot be type arguments)
`map.put(101, "Rahul")`	Stores the pair; the argument order is *key, then value*
`map.get(101)`	Looks up the key `101` and returns its value, `"Rahul"`
`{101=Rahul, ...}`	Maps print in curly braces with `key=value`

Common `HashMap` methods

Method	Purpose
`put()`	Insert key-value
`get()`	Retrieve value
`remove()`	Delete entry
`containsKey()`	Search key
`keySet()`	Get keys
`values()`	Get values

This supplementary example uses the remaining methods. `containsKey` answers yes/no, `keySet` and `values` return the keys and the values separately, and a second `put` with an existing key *replaces* that key's value instead of adding a new pair. Asking `get` for a key that is not present returns `null`.

```
import java.util.*;

class MapMethods {
  public static void main(String args[]) {
    HashMap<Integer, String> map =
        new HashMap<>();
    map.put(101, "Rahul");
    map.put(102, "Amit");
    map.put(103, "Neha");
    System.out.println(map.containsKey(102));
    System.out.println(map.keySet());
    System.out.println(map.values());
    map.put(102, "Amit Kumar");
    System.out.println(map);
    map.remove(101);
    System.out.println(map);
    System.out.println(map.get(999));
  }
}
```

Output:

```
true
[101, 102, 103]
[Rahul, Amit, Neha]
{101=Rahul, 102=Amit Kumar, 103=Neha}
{102=Amit Kumar, 103=Neha}
null
```

> **Note for Review:** Uniqueness applies to **keys only**. Re-using a key overwrites its value; re-using a value is perfectly fine. Supplementary: `TreeMap`, the other example named in the lecture, keeps its keys in sorted order, just as `TreeSet` does for elements.

---

9. Iterators

The problem. A `Set` has no positions, so `get(i)` is useless. We still want to visit every element, one by one, in the same way for *any* collection.

The analogy. A librarian walking along a shelf with a bookmark, checking whether another book exists, taking it, and moving on.

An **`Iterator`** traverses the elements of a collection one by one. It is an interface, `java.util.Iterator`, with three methods:

Method	Description
`hasNext()`	Checks whether a next element exists
`next()`	Returns the next element
`remove()`	Removes the element last returned by `next()`

The program fills an `ArrayList`, asks it for an iterator with `list.iterator()`, and loops while `hasNext()` is true, printing each `next()`. The iterator's type argument `<String>` matches the list's, so `next()` returns a `String` with no cast needed.

```
import java.util.*;

class IteratorDemo {
  public static void main(String args[]) {
    ArrayList<String> list =
        new ArrayList<>();
    list.add("Java");
    list.add("Python");
    list.add("C++");

    Iterator<String> itr =
        list.iterator();

    while (itr.hasNext()) {
      System.out.println(itr.next());
    }
  }
}
```

Output:

```
Java
Python
C++
```

Step	Cursor position	**`hasNext()`**	**`next()` returns**
Start	Before "Java"	`true`	
1	After "Java"	`true`	`Java`
2	After "Python"	`true`	`Python`
3	After "C++"	`false`	`C++`
4		`false` → loop ends	

Common mistakes

- Calling `next()` without checking `hasNext()` first throws a `NoSuchElementException` once the elements run out.
- Removing from the list directly (`list.remove(...)`) while iterating usually throws a `ConcurrentModificationException`. Use the iterator's own `remove()` instead.

```
Iterator<String> itr = list.iterator();
while (itr.hasNext()) {
  if (itr.next().equals("Python")) {
    itr.remove();            // safe removal
  }
}
System.out.println(list);    // [Java, C++]
```

9.1 `ListIterator`

`ListIterator` allows both **forward traversal** and **backward traversal**. It is obtained with `list.listIterator()` and works only for lists.

The first loop moves the cursor forward to the end using `hasNext()` and `next()`. The second loop then walks the same iterator back to the start using `hasPrevious()` and `previous()`. The backward loop prints something only because the cursor is already at the end after the forward loop.

```
ListIterator<String> itr =
    list.listIterator();

while (itr.hasNext()) {
  System.out.println(itr.next());
}

while (itr.hasPrevious()) {
  System.out.println(itr.previous());
}
```

For the list `[Java, Python, C++]` this prints `Java`, `Python`, `C++` going forward, and then `C++`, `Python`, `Java` coming back.

> **Note for Review:** A fresh `ListIterator` starts *before the first element*, so `hasPrevious()` is `false` until you have moved forward. Supplementary: the enhanced `for` loop (`for (String s : list)`) uses an `Iterator` behind the scenes.

---

10. Choosing the Right Collection

Selecting a class is a requirements question. Ask what your data must do.

Requirement	Choose
Ordered data, duplicates allowed, fast access by index	`ArrayList`
Ordered data with frequent insertion/deletion (for example at the front)	`LinkedList`
Unique elements, order does not matter, fastest	`HashSet`
Unique elements in sorted order	`TreeSet`
Unique elements in insertion order	`LinkedHashSet`
Items processed first-in, first-out (scheduling, service desks)	`Queue` (for example a `LinkedList`)
Look up values by a unique key	`HashMap`

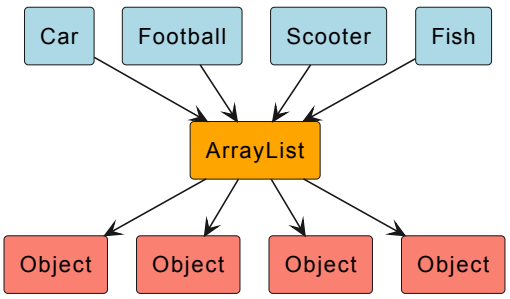
11. Generics

The problem. A teacher keeps a list of marks. By accident someone adds the text `"absent"` into it. The program compiles, runs, and then crashes when it tries to add the marks up. The mistake was possible because the list never checked what it was given.

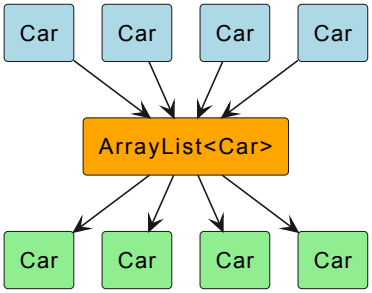
The analogy. An unlabelled storage box accepts anything, and you only find the wrong item when you reach in. A box labelled "Marks only" refuses wrong items at the moment you try to put them in.

Generics allow classes and methods to work with different data types while providing **compile-time type safety**.

Without generics an `ArrayList` accepts references to any kind of object: a car, a football, a scooter and a fish all go in. Whatever goes in, every element comes out as a plain `Object` reference, so the program must remember what each one really is.



With generics the list is declared as `ArrayList<Car>`. Only references to `Car` objects can go in, and every element comes out as a reference of type `Car`. The compiler enforces this, so no wrong type can slip in and no cast is needed on the way out.



The upper half is the pre-generics style (a *raw type*): `list.add(10)` is accepted next to a string, and the mistake only explodes when the code casts the wrong element at run time. In the lower half the type argument `<String>` lets the compiler reject `names.add(10)` before the program ever runs.

```
// Before generics (raw type)
ArrayList list = new ArrayList();
list.add("Java");
list.add(10);           // accepted!
String s = (String) list.get(1);
                        // runtime error

// Using generics
ArrayList<String> names =
    new ArrayList<>();
names.add("Java");
// names.add(10);       // INVALID:
                        // compile-time error
```

Without generics	With generics
No type checking	Type checked by the compiler
Runtime errors possible (`ClassCastException` when the cast is wrong)	Wrong-type code is rejected at compile time
Explicit casts needed when reading	No casting needed

Advantages of generics: type safety, no type casting, **code reuse**, better readability and errors found at compile time.

> **Note for Review:** The invalid line `names.add(10)` is a *compile-time* error: the program will not compile, and the exact wording of the message depends on the compiler. The raw-type cast error is a *run-time* error: the program compiles (with a warning) and fails only when that line executes.

11.1 Generic class

`Box<T>` is a generic class. `T` is a *type parameter*, a placeholder that is replaced by a real type when the class is used. In `Box<Integer>`, every `T` becomes `Integer`, so `set` accepts only an `Integer` and `get` returns an `Integer`. One class therefore serves every data type.

```
class Box<T> {
  T value;

  void set(T value) {
    this.value = value;
  }

  T get() {
    return value;
  }
}

class BoxDemo {
  public static void main(String args[]) {
    Box<Integer> b = new Box<>();
    b.set(100);
    System.out.println(b.get());
  }
}
```

Output:

```
100
```

General pattern

```
class ClassName<T> {
    T field;                 // a field of the placeholder type
    void method(T param) { } // parameter of the placeholder type
    T other() { return field; }
}
```

Part	Meaning
`class Box<T>`	Declares a generic class with one type parameter named `T`
`T value;`	A field whose type is decided by the user of the class
`void set(T value)`	Parameter named `value` of type `T`
`this.value = value;`	`this.value` is the *field*; plain `value` is the *parameter*. Without `this.` the assignment would copy the parameter onto itself
`Box<Integer> b = new Box<>();`	Creates a box that holds an `Integer`; `<>` infers `Integer`

Step	Operation	Result
1	`new Box<>()`	A box object; `T` is `Integer`; `value` is `null`
2	`b.set(100)`	`100` is wrapped as an `Integer` and stored in the field
3	`b.get()`	Returns the `Integer` `100`
4	`println`	Prints `100`

> **Note for Review:** By convention `T` stands for *Type*; `E` (Element), `K` (Key) and `V` (Value) are also common, which is why `Map<K, V>` has two. A type argument must be a class, never a primitive: `Box<int>` is invalid, `Box<Integer>` is valid.

11.2 Generic method

A method can have its own type parameter, independent of any class.

The `<T>` written *before the return type* declares a type parameter for this method alone. The method `display` then accepts a value of any type. The two calls pass an `int` and a `String`, and the compiler infers `T` from the argument each time.

```
class GenericDemo {
  static <T> void display(T data) {
    System.out.println(data);
  }

  public static void main(String args[]) {
    display(100);
    display("Java");
  }
}
```

Output:

```
100
Java
```

Part	Meaning
`static`	The method belongs to the class, so `main` can call it directly
`<T>`	Declares the method's type parameter (placed *before* the return type)
`void`	Return type: nothing is returned (the method prints instead)
`T data`	Parameter of the placeholder type

	Generic class	Generic method
Where `<T>` appears	After the class name: `class Box<T>`	Before the return type: `static <T> void display(T data)`
Scope of `T`	The whole class	That method only

> **Note for Review (supplementary):** Comparing two values of type `T` with `>` does not compile, because an arbitrary `T` might not be comparable. For tasks like "find the maximum of three values" you restrict the type: `static <T extends Comparable<T>> T max(T a, T b, T c)`, then compare with `a.compareTo(b)`.

12. Wrapper Classes

The problem. Collections and generics work with *objects* only. But `int`, `double` and `char` are primitives, not objects. So `ArrayList<int>` is invalid.

The analogy. A courier delivers only boxed parcels. A loose coin (a primitive) must be put into a box (a wrapper object) before it can be posted.

Wrapper classes convert primitive data types into objects (primitive → wrapper). Collections store only objects, so:

```
ArrayList<int> list;       // INVALID: primitive type argument
ArrayList<Integer> list;   // valid: wrapper class
```

Primitive	Wrapper
`int`	`Integer`
`char`	`Character`
`double`	`Double`
`float`	`Float`
`boolean`	`Boolean`
`byte`	`Byte`
`short`	`Short`
`long`	`Long`

The wrapper name is the primitive name with a capital letter, except the two irregular pairs `int` → `Integer` and `char` → `Character`.

This program wraps a primitive manually. `Integer.valueOf(x)` is a static method of the `Integer` class that takes the primitive `int` in `x` and returns an `Integer` object holding the same value. The object is then printed.

```
class WrapperDemo {
  public static void main(String args[]) {
    int x = 10;
    Integer obj = Integer.valueOf(x);
    System.out.println(obj);
  }
}
```

Output:

```
10
```

Part	Meaning
`int x = 10;`	A primitive variable holding the value itself
`Integer obj`	A reference variable of the wrapper class
`Integer.valueOf(x)`	Class name, a dot, and the static conversion method
`println(obj)`	Prints the value held by the object

12.1 Autoboxing and unboxing

Writing `Integer.valueOf(...)` every time is tedious, so Java converts automatically.

	Direction	Example	What Java does
Autoboxing	Primitive → Object	`Integer obj = 100;`	`int` → `Integer`
Unboxing	Object → Primitive	`int x = obj;`	`Integer` → `int`

The first two lines show a boxing conversion and an unboxing conversion. The list then shows why this matters: `nums.add(5)` passes an `int` where an `Integer` is required (autoboxing), and `int first = nums.get(0)` reads an `Integer` into an `int` (unboxing). The program prints `x + first`, which is `100 + 5`.

Output:

```
105
```

```
import java.util.*;

class BoxingDemo {
  public static void main(String args[]) {
    Integer obj = 100;  // autoboxing
    int x = obj;        // unboxing

    ArrayList<Integer> nums =
        new ArrayList<>();
    nums.add(5);        // int -> Integer
    int first = nums.get(0);
                        // Integer -> int
    System.out.println(x + first);
  }
}
```

> **Note for Review (supplementary):** Two traps. First, unboxing a `null` wrapper throws a `NullPointerException`. Second, compare wrapper objects with `.equals()`, not `==`: small values (-128 to 127) are cached, so `Integer a = 127, b = 127;` gives `a == b` as `true`, but `Integer c = 1000, d = 1000;` gives `c == d` as `false` (while `c.equals(d)` is `true`). This is also why `remove(1)` and `remove(Integer.valueOf(1))` behave differently on an `ArrayList<Integer>`.

13. Multithreading and the Thread Life Cycle

The problem. A web browser must download a file, play a video, load a page and react to your typing at the same time. If the program did these jobs one after another, the window would freeze whenever one of them was busy.

The analogy. A restaurant kitchen. The kitchen is the **process** (one building, one shared pantry). The cooks are **threads**: each cook works independently, but all of them share the same pantry. Where the analogy stops: a real kitchen has as many cooks as it hires, while the number of threads that can run truly at the same instant is limited by the CPU's cores.

A **thread** is a lightweight sub-process that represents an independent path of execution within a program. A program can contain multiple threads executing simultaneously. In the browser, each task can be a separate thread: `T1` downloads files, `T2` plays video, `T3` loads web pages, `T4` handles user input.

Three headline advantages of multithreading: it **allows multiple operations to be performed at once**, it **saves time** when several operations are run together, and the threads are **independent of each other**.

> **Note for Review (supplementary):** *Concurrent* means several tasks make progress during the same period (the CPU may switch rapidly between them). *Parallel* means they literally run at the same instant on different cores. The lecture uses the two words loosely; for the mechanics in this chapter the difference does not matter.

13.1 Process versus thread

Any running application program is a **process**.

Process	Thread
Independent execution unit	Smallest execution unit
Requires more memory	Requires less memory
Communication is slower	Communication is faster
Has its own memory space	Shares the memory of its process

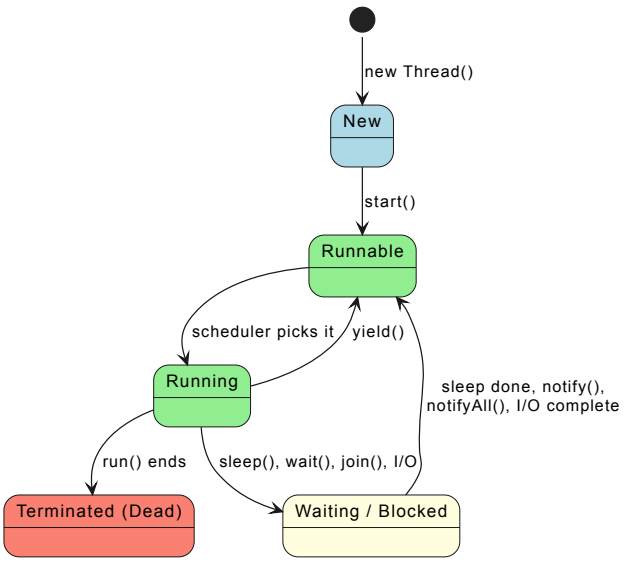
13.2 Advantages of multithreading

- Improved application performance
- Better CPU utilization
- Faster execution
- Improved responsiveness
- Parallel task execution

13.3 Thread life cycle

A thread passes through different states during its execution. The state names matter, and so does *which method causes each move between states*: `start()`, `run()`, `yield()`, `sleep()`, `wait()`, `notify()` and `notifyAll()`.

The diagram shows the life of one thread. A thread is created in the *New* state and `start()` makes it *Runnable* (ready for execution). The scheduler moves it to *Running* when the CPU executes it; `yield()` can send it back to Runnable. From Running it can enter *Waiting/Blocked* (through `sleep()`, `wait()`, `join()` or while waiting for I/O) and returns to Runnable when the wait ends or `notify()` / `notifyAll()` wakes it. When `run()` ends the thread is *Terminated* (Dead).



Java defines thread states using `Thread.State`. The main states are:

State	Meaning	Typical trigger or example
New	Thread object is created but not started	`Thread t = new Thread();`
Runnable	Thread is ready for execution	`t.start();`
Running	The CPU is executing the thread	Scheduler selects it
Waiting / Blocked	Thread waits for a resource or another thread	`sleep()`, `wait()`, `join()`
Terminated (Dead)	Thread execution is complete	The `run()` method ends

> **Note for Review:** This five-state picture is the *conceptual* model. The actual Java enum `Thread.State` has the constants `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING` and `TERMINATED`. There is no separate `RUNNING`: `RUNNABLE` covers both "ready" and "running". Likewise, the lecture's single "Waiting/Blocked" box splits into `WAITING`, `TIMED_WAITING` (for example `sleep`) and `BLOCKED` (waiting to enter a synchronized block).

14. Creating and Controlling Threads

Java provides **two ways** to create a thread:

1. **Extending the `Thread` class**
2. **Implementing the `Runnable` interface**

In both ways the work the thread must do is written in a method named `run()`.

14.1 Method 1: extending `Thread`

`MyThread` is a subclass of `Thread` that overrides `run()` with the code the thread should execute. In `main`, `new MyThread()` puts the thread in the *New* state and `t.start()` makes it *Runnable*. The JVM then calls `run()` on the new thread, which prints the message.

```
class MyThread extends Thread {
  public void run() {
    System.out.println(
        "Thread is running");
  }

  public static void main(
      String args[]) {
    MyThread t = new MyThread();
    t.start();
  }
}
```

Output:

```
Thread is running
```

General pattern

```
class ClassName extends Thread {
    public void run() {
        // code the thread executes
    }
}
// ... in main:
ClassName t = new ClassName();   // New state
t.start();                       // Runnable; JVM then calls run()
```

Part	Meaning
`extends Thread`	Inherits the thread machinery from the `Thread` class
`public void run()`	Overrides `Thread.run()`; holds the thread's code. It must be `public` and `void`
`new MyThread()`	Creates the thread object (state: New). No new execution path exists yet
`t.start()`	Registers the thread for execution (state: Runnable); the JVM later calls `run()`

> **Note for Review:** The classic mistake is calling `t.run()` instead of `t.start()`. `t.run()` is just an ordinary method call executed by the *current* thread; no new thread is created. Only `start()` creates a new path of execution. Also, `start()` may be called only once on a thread object.

14.2 Important `Thread` methods

Method	Description
`start()`	Starts execution
`run()`	Contains thread code
`sleep()`	Pauses execution
`join()`	Waits for another thread
`getName()`	Gets thread name
`setName()`	Sets thread name

This supplementary program uses the other methods in the table. `setName` names the thread, `getName` reads it, and `Thread.sleep(500)` pauses the thread for half a second. `w.join()` makes `main` wait until the worker terminates, so the final line always prints last.

```
class Worker extends Thread {
  public void run() {
    System.out.println(
        getName() + " started");
    try {
      Thread.sleep(500);
    } catch (InterruptedException e) {
    }
    System.out.println(
        getName() + " finished");
  }

  public static void main(String[] args)
      throws InterruptedException {
    Worker w = new Worker();
    w.setName("Worker-A");
    w.start();
    w.join();
    System.out.println("main continues");
  }
}
```

Output:

```
Worker-A started
Worker-A finished
main continues
```

> **Note for Review:** `sleep()` and `join()` can throw a *checked* exception, `InterruptedException`, so the compiler insists that you either catch it (`try` / `catch`) or declare it (`throws InterruptedException`). Without one of the two, the code does not compile.

14.3 Method 2: implementing `Runnable`

`Runnable` provides a better design because **Java supports single inheritance**: a class that already extends another class cannot also extend `Thread`, but it can still implement `Runnable`.

The analogy. The `Runnable` object is a *job card* (what to do). The `Thread` object is the *worker* who carries it out. Method 1 makes the job card and the worker the same object; Method 2 separates them.

`MyTask` implements `Runnable`, so it supplies the job in `run()` but is not itself a thread. In `main`, the task object is passed to the `Thread` constructor, which creates the worker thread that will run it. `t.start()` then makes the thread run the task's `run()` method.

```
class MyTask implements Runnable {
  public void run() {
    System.out.println(
        "Runnable Thread");
  }

  public static void main(
      String args[]) {
    MyTask task = new MyTask();
    Thread t = new Thread(task);
    t.start();
  }
}
```

Output:

```
Runnable Thread
```

	Extending `Thread`	Implementing `Runnable`
Keyword	`extends Thread`	`implements Runnable`
Starting	`new MyThread().start()`	`new Thread(task).start()`
Can the class extend another class?	No (single inheritance)	Yes
Design	Task and worker are one object	Task and worker are separate

14.4 Creating multiple threads

Two objects of the same thread class are created and started one after another. Each thread's `run()` prints the numbers 1 to 5, so ten numbers appear in total. The threads execute independently, so how the two sequences interleave can differ from run to run.

Output may vary because threads execute independently. Two possible outputs:

Possible run A	Possible run B
`1 2 3 4 5` then `1 2 3 4 5`	`1 1 2 2 3 3 4 4 5 5` (interleaved)

```
class ThreadDemo extends Thread {
  public void run() {
    for (int i = 1; i <= 5; i++) {
      System.out.println(i);
    }
  }

  public static void main(
      String args[]) {
    ThreadDemo t1 = new ThreadDemo();
    ThreadDemo t2 = new ThreadDemo();

    t1.start();
    t2.start();
  }
}
```

What is *guaranteed* is that each thread prints its own numbers in the order 1 to 5. What is *not* guaranteed is which thread prints when. Printing `getName()` together with `i` makes the interleaving visible (the default names are `Thread-0`, `Thread-1`).

> **Note for Review:** Starting `t1` first does not mean it finishes first, or even prints first. The thread scheduler decides, and your program must not depend on its choices.

15. Thread Synchronization

The problem. Two people share one bank account holding ₹1000. At the same moment each tries to withdraw ₹800. Each checks the balance, sees ₹1000, and approves. Both withdrawals succeed and the account goes negative. This happens whenever **multiple threads access shared resources simultaneously**: data inconsistency may occur.

Even a plain `count++` is risky, because it is really three steps: *read* the value, *add one*, *write* it back.

Step	Thread T1	Thread T2	Shared `count`
1	reads `count` (0)		0
2		reads `count` (0)	0
3	writes 1		1
4		writes 1	1

Two increments were performed, yet the final value is `1` instead of `2`: one update was *lost*. The shared part of the code is the **critical section**.

Synchronization allows only one thread to access a critical section at a time. It is achieved with the **`synchronized` keyword**. While one thread is inside, the others wait (in the Blocked state) until it leaves.

15.1 Synchronized method

```
class Counter {
  int count = 0;

  synchronized void increment() {
    count++;
  }
}

class IncrementTask implements Runnable {
  Counter counter;

  IncrementTask(Counter counter) {
    this.counter = counter;
  }

  public void run() {
    for (int i = 0; i < 100000; i++) {
      counter.increment();
    }
  }
}

class CounterDemo {
  public static void main(String[] args)
      throws InterruptedException {
    Counter counter = new Counter();
    Thread t1 =
        new Thread(new IncrementTask(counter));
    Thread t2 =
        new Thread(new IncrementTask(counter));
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println(counter.count);
  }
}
```

Making `increment()` `synchronized` means a thread must hold the object's *lock* to run it. Only one thread can execute `increment()` on the same `Counter` object at a time. The driver below starts two threads that share one `Counter` and waits for both with `join()`, so the printed total is reliably 200000.

Output:

```
200000
```

Only `Counter` is the lecture's code; `IncrementTask` and `CounterDemo` are supplementary so that the example can run.

Part	Meaning
`synchronized void increment()`	`synchronized` is a method modifier (like `public`; it can be combined, as in `public synchronized void`). One thread at a time per object
`IncrementTask(Counter counter)`	Constructor that stores the reference, so both tasks work on the *same* `Counter` object
`t1.join(); t2.join();`	`main` waits for both threads before reading `count`

> **Note for Review:** Synchronization works because both threads compete for the **same lock**, the lock of the one shared `Counter` object. If each thread had its own `Counter`, there would be nothing shared and nothing to protect. If you remove the word `synchronized`, the total *may* come out below 200000, but it is not guaranteed to fail on every run, which is exactly what makes such bugs dangerous.

15.2 Synchronized block

Instead of synchronizing the whole method, synchronize only the lines that need protection.

In a synchronized block, the expression in parentheses is the *lock object*. Here `this` means the current object, so only one thread at a time can be inside the block for that object. Code outside the block can still run in parallel.

```
void increment() {
  // work that needs no lock
  synchronized (this) {
    count++;
  }
}
```

Advantages: better performance and a smaller critical section (threads wait only for the lines that truly need protection).

	Synchronized method	Synchronized block
Syntax	`synchronized void increment() { ... }`	`synchronized (this) { ... }` inside a method
What is locked	Whole method body	Only the block
Lock object	Current object (`this`)	Any object you name

16. Inter-thread Communication

The problem. Synchronization stops threads from clashing, but sometimes threads must *cooperate*. A consumer must not take an item that has not been produced yet, and a producer must not overwrite an item nobody has used yet.

Threads communicate with each other using three methods:

- `wait()` : **releases the lock** and places the thread into the **waiting state**.
- `notify()` : **wakes up one** waiting thread.
- `notifyAll()` : **wakes up all** waiting threads.

These methods belong to the **`Object` class** (not `Thread`), so every object has them.

> **Note for Review:** `wait()`, `notify()` and `notifyAll()` must be called from inside synchronized code (a synchronized method or block) on the same object; otherwise Java throws an `IllegalMonitorStateException`. Compare with `sleep()` : a sleeping thread **keeps** the lock, while a waiting thread **releases** it, which is what lets another thread enter and do the work it is waiting for.

16.1 The producer-consumer problem

The analogy. A tiffin counter with room for exactly one dish. The cook (producer) may place a dish only when the counter is empty; the waiter (consumer) may pick one up only when a dish is there. Each tells the other when the counter's state changes.

The shared object has a value and a flag `available` : `false` means the counter is empty.

`produce` is synchronized. While an item is still available (not yet consumed) the producer waits. Once the counter is empty it stores the value, sets `available` to `true`, prints, and calls `notify()` to wake a waiting consumer. `consume` mirrors this: it waits while nothing is available, then consumes, sets `available` to `false` and notifies.

The class `Shared` alone is not a program, so this supplementary driver adds a producer thread and a consumer thread that share one `Shared` object. The producer produces the values 1 to 3 and the consumer consumes three times. Starting both threads lets the `wait` / `notify` handshake force them to alternate.

```
class Shared {
  int value;
  boolean available = false;

  synchronized void produce(int v) {
    while (available) {
      try {
        wait();
      } catch (Exception e) { }
    }
    value = v;
    available = true;
    System.out.println("Produced: " + value);
    notify();
  }

  synchronized void consume() {
    while (!available) {
      try {
        wait();
      } catch (Exception e) { }
    }
    System.out.println("Consumed: " + value);
    available = false;
    notify();
  }
}
```

```
class Producer extends Thread {
  Shared shared;
  Producer(Shared shared) {
    this.shared = shared;
  }
  public void run() {
    for (int i = 1; i <= 3; i++)
      shared.produce(i);
  }
}

class Consumer extends Thread {
  Shared shared;
  Consumer(Shared shared) {
    this.shared = shared;
  }
  public void run() {
    for (int i = 1; i <= 3; i++)
      shared.consume();
  }
}

class ProducerConsumerDemo {
  public static void main(String[] args) {
    Shared shared = new Shared();
    new Producer(shared).start();
    new Consumer(shared).start();
  }
}
```

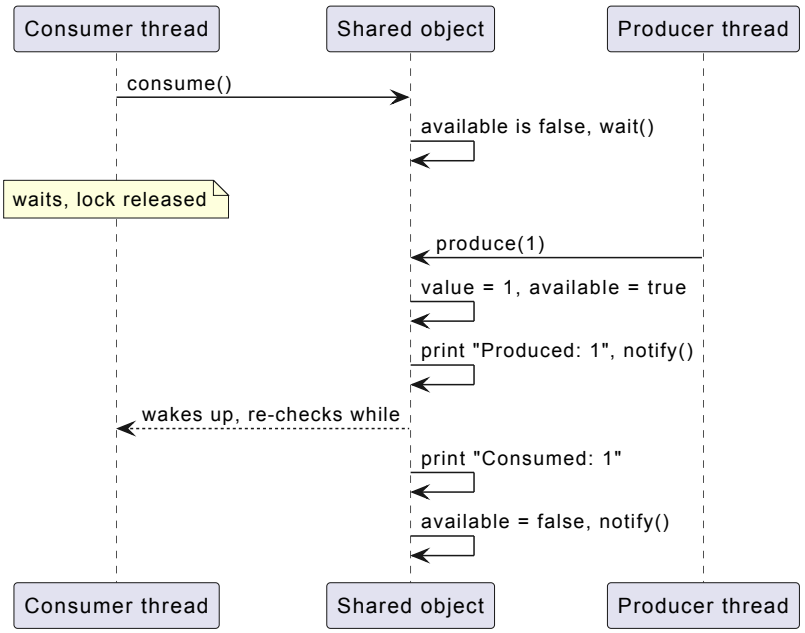
Output:

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
```

The lines alternate in strict order, whichever thread happens to run first, because the flag forbids producing twice in a row or consuming twice in a row.

Execution trace (consumer arrives first)

Suppose the consumer thread runs before anything has been produced. It finds `available` false and calls `wait()`, releasing the lock. The producer can now enter, store the value, print and `notify()`. The consumer wakes, re-checks the condition and consumes. In thread-state terms, the consumer moved Running → Waiting → Runnable → Running.



Part	Why it is written this way
`synchronized` on both methods	Only one of produce/consume runs at a time; also required to call `wait()` / `notify()`
`while (available)` / `while (!available)`	The condition is **re-checked after waking**; a woken thread may find that the situation has changed again
`try { wait(); } catch (Exception e) { }`	`wait()` throws a checked `InterruptedException`. Leaving the `catch` empty is a lecture simplification; real code should handle it
`notify()`	Tells the waiting partner that the state changed

> **Note for Review:** Use `while`, not `if`, around `wait()`. With `if`, a woken thread goes straight ahead without re-testing the flag, which breaks as soon as more than one consumer exists. Use `notifyAll()` when several threads may be waiting for different conditions.

---

17. Summary and Lecture Exercises

Practice exercises: collections and generics

1. Create an `ArrayList` to store student names and perform add, remove and search operations.
2. Implement a `LinkedList`-based queue system for customer service management.
3. Remove duplicate elements from a list using `HashSet`.
4. Store employee records using `HashMap` with employee ID as the key.
5. Sort student marks using `TreeSet`.
6. Traverse an `ArrayList` using `Iterator` and `ListIterator`.
7. Create a generic class to store and retrieve different data types.
8. Implement a generic method to find the maximum of three values.
9. Demonstrate autoboxing and unboxing using wrapper classes.
10. Develop a mini contact management system using `HashMap`.

Practice exercises: multithreading

1. Create a Java program that creates two threads: one prints even numbers and another prints odd numbers.
2. Implement a multithreaded bank account system using synchronization.
3. Develop a producer-consumer program using `wait()` and `notify()`.

> **Further reading:** *Java: The Complete Reference* has many worked examples and exercises on collections and multithreading.

Practice Questions

Question 1: Library desk (design and justify). A library keeps its books in a collection where each book has a unique integer ID and a title. When a book is borrowed and another student asks for it, that student joins a waiting list, and when the book is returned it goes to the student who has been waiting longest. Write a class `LibraryDesk` with the methods `addBook(int id, String title)`, `requestBook(String student, int id)` and `returnBook(int id)`. Choose the collection types for the catalogue and for each waiting list, explain why each is suitable and why other options (for example an `ArrayList` for the catalogue) would be inferior, and give the generic type arguments you use.

Question 2: Predict the output. Trace the program below and write the exact output. For each of the two `remove` calls, state which overload is selected and why.

Question 3: Overdraft race (threads and synchronization). A `BankAccount` object holds a balance of 1000 and has an unsynchronized method `withdraw(int amount)` that first checks `balance >= amount` and then subtracts the amount. Two threads share the *same* account and each withdraws 800. (a) Write a step-by-step interleaving, in the style of a trace table, that leaves the balance at -600. (b) Fix the class and explain exactly which part of `withdraw` is the critical section. (c) Explain what changes, and why, if each thread creates its own `BankAccount` object instead. (d) Decide whether a synchronized block could replace the synchronized method here, and justify your answer.

Question 4: Find and classify the errors (generics and wrappers). The code below contains several mistakes. For each, state whether it is a compile-time error, a run-time exception or a logical error, explain what Java sees, and write the corrected version. Then give the output of your corrected program.**Question 5: Two consumers (thread states and communication).** In the producer-consumer program, replace both `while` loops by `if` statements and run **one producer and two consumers** that share the same `Shared` object. (a) Describe a sequence of events in which a consumer prints a value that has already been consumed. (b) For each of the three threads, list the thread states it passes through during your sequence and the method call that causes each change. (c) Explain why changing `if` back to `while` repairs the problem, and whether `notify()` could still leave a thread waiting forever in a program with several producers and several consumers.

```
// Question2
import java.util.*;

class Trace {
  public static void main(String args[]) {
    ArrayList<Integer> scores = new ArrayList<>();
    scores.add(1);
    scores.add(40);
    scores.add(1);
    scores.add(25);

    scores.remove(1);
    scores.remove(Integer.valueOf(1));
    System.out.println(scores);

    HashSet<Integer> unique = new HashSet<>();
    TreeSet<Integer> sorted = new TreeSet<>();
    Iterator<Integer> itr = scores.iterator();
    while (itr.hasNext()) {
      int s = itr.next();
      unique.add(s);
      sorted.add(s);
    }
    unique.add(25);
    unique.add(40);
    sorted.add(5);
    System.out.println(unique.size() + " " + sorted);
  }
}
```

```
// Question 4
class Box<T> {
  T value;
  void set(T value) { value = value; }
  T get() { return value; }
}

class Test {
  public static void main(String args[]) {
    ArrayList<int> marks = new ArrayList<>();
    marks.add("85");
    Box<String> b = new Box<>();
    b.set(100);
    int total = marks.get(0) + b.get();
    System.out.println(total);
  }
}
```